Deliverable Contracts: A Deterministic Governance Layer for Long-Horizon Agent Tasks

Anonymous Author(s)
Long-Horizon Agent Governance Research Project

Abstract

Long-horizon autonomous agent runs are typically governed by trusting the agent's own self-report that a task is "done." We show, on 300 real, publicly published coding-agent submissions to SWE-bench Lite, that this status-quo governance signal has zero ability to distinguish a sound deliverable from an unsound one: every instance the agent's own harness reported as "submitted" is treated identically by this baseline, regardless of whether the underlying patch is structurally coherent. We introduce GATEKEEP, a deterministic, offline, sub-millisecond *deliverable contract* checker: a small declarative schema for what a long-horizon task must produce, plus a checker that verifies the actual artifact state against that schema without any model call. Applied to the same 300 real agent submissions and scored against the official SWE-bench evaluation harness's ground truth, GATEKEEP achieves perfect recall (1.00) and improves precision over the status-quo baseline (0.2438 vs. 0.2404) by flagging four deliverables as structurally unsound before any test suite is run, at zero added false negatives. We report the full precision/recall/F1 table, document two real false-positive bugs we found and fixed during the experiment, and discuss honestly what a cheap deterministic check can and cannot catch. The system, benchmark scoring code, and raw logs are publicly released.

1 Introduction

Long-horizon agent tasks — multi-hour autonomous coding sessions, research tasks, or any workflow where an LLM-driven agent operates for extended periods with multiple intermediate steps before producing a final set of deliverables — have a specific, under-discussed failure mode that we call *deliverable drift*: the task specifies N required outputs, and over the course of a long run the agent silently drops, truncates, or fabricates one or more of them while still reporting overall success.

The standard way agent harnesses today decide whether to trust a deliverable is to trust the agent's own completion signal: an exit code, an "I'm done" message, or a non-null artifact in the expected location. We call this *status-quo governance*. It is cheap and ubiquitous, but it has an obvious weakness: it asks the agent to grade its own homework.

This paper makes two contributions:

1. **A measurement.** Using 300 real, publicly available agent submissions to SWE-bench Lite (SWE-agent driving Claude 3.5 Sonnet, published by the SWE-bench project), we measure how often status-quo governance — "a non-empty submission with a clean exit status" — is wrong about deliverable soundness, against real ground truth from the official SWE-bench evaluation harness. We find it has *zero* discriminative power on this population: every

Preprint. Under review.

instance with a non-null, cleanly-submitted patch is classified identically as "shippable," whether or not the patch actually resolves the issue (Table 1).

2. **A system.** GATEKEEP, a deterministic deliverable governance layer. Long-horizon task owners declare a *deliverable contract* (a YAML file: what must exist, in what shape, with what content properties) and run a sub-millisecond, offline checker against it. We show this catches a measurable subset of unsound deliverables that status-quo governance misses, with no added false negatives, using only deterministic, reproducible checks — no LLM call, no API key, no network dependency.

We are explicit about scope: GATEKEEP is not a smarter SWE-bench solver, and a deterministic syntactic/structural check cannot, by construction, verify semantic correctness (whether the patch actually fixes the bug). What it verifies is a necessary but not sufficient condition: did the agent produce something that even has the right shape to be correct? Section 6 discusses this honestly, including two real false-positive bugs we discovered and fixed while running the experiment in this paper.

2 Related Work

Agent benchmarks. SWE-bench [Jimenez et al., 2024] evaluates whether an LM-driven agent can resolve real GitHub issues, scored by running the project's test suite against the agent's patch in a sandboxed container. SWE-bench Lite is a 300-instance subset used for faster iteration. The SWE-bench project also maintains a public `experiments` repository of raw submissions (trajectories, patches, and evaluation reports) from many agents evaluated against the benchmark; we use one such public, externally-produced submission (SWE-agent [Yang et al., 2024] driving Claude 3.5 Sonnet) as the substrate for our measurement, rather than producing new rollouts ourselves.

LLM-as-judge evaluation. A large body of agent evaluation work uses an LLM to grade another LLM's output against a rubric. This is expressive but non-deterministic and requires a model call (cost, latency, API access) per judgment. GATEKEEP takes the opposite design point: every check is a pure function over the filesystem, so the same contract gives the same verdict for anyone, forever, independent of model version or API availability. We do not claim this is strictly better for every use case — only that it is a different, complementary point in the design space, useful precisely when reproducibility and zero marginal cost matter more than rubric expressiveness.

CI/build verification. Software engineering has long used deterministic gates (linters, type checkers, test runners) to verify build artifacts before merge. GATEKEEP applies the same philosophy to agent task *deliverables* broadly (not just code): a declarative contract plus a fast, auditable checker, designed to be dropped into existing CI pipelines (we ship a GitHub Actions wrapper).

3 The gatekeep System

3.1 Deliverable contracts

A contract (Listing 1) declares a list of named *deliverables*. Each deliverable has a severity (`required` or `advisory`) and a list of *checks*. A run passes iff every `required` deliverable's checks all pass; `advisory` failures are reported but never block.

Listing 1: A deliverable contract.

```
name: my-project-deliverables
deliverables:
  - id: api-patch
    description: "the agent's code change is real"
    severity: required
    checks:
      - kind: valid_unified_diff
        path: "patch.diff"
        forbid_test_only: true
  - id: readme
    severity: required
    checks:
```

```
  - kind: min_words
    path: "README.md"
    min_words: 50
  - kind: no_placeholder_text
    path: "README.md"
```

3.2 Check kinds

The current engine implements eight deterministic check kinds: `file_exists` (glob match with a minimum count), `min_size` (byte-size floor), `min_words` (word-count floor), `no_placeholder_text` (regex bank for TODO/FIXME/lorem ipsum/"not implemented"/etc., with an `allow`-list escape hatch and an optional diff-aware mode that scans only *added* lines), `json_valid`, `regex_required`, `valid_unified_diff` (structural diff validation: real `@@` hunks, minimum file count, optional "no test-only patches" rule), and `forbid_glob` (ban stray scratch files). Every check is a small, independently testable pure function; adding a new kind is one function plus a registry entry.

3.3 Two install paths, kept in parity

GATEKEEP ships as (a) a pip-installable package with a `gatekeep` console entry point, and (b) a single, zero-dependency file (`gatekeep_single.py`, stdlib only, including a self-contained YAML-subset parser) that a stranger can `curl` and run with no install step. Both paths read the identical contract format and are kept in lock-step by an automated parity test suite (`test_single_file_parity.py`) that runs the same contracts through both implementations and asserts identical verdicts. The full test suite (20 tests at submission time) covers both engines plus the YAML subset parser, including regression tests for every bug found during this paper's experiment (Section 6).

3.4 Self-application

As a qualitative check, we wrote a GATEKEEP contract describing this project's own five required deliverables (deployable system, paper, public-benchmark A/B, marketing plan, landing page) and ran GATEKEEP against this very repository before submission. This dogfooding step is what surfaced two of the bugs fixed in Section 6: the contract's own README initially failed its own placeholder-text check because the README *documents* the words "TODO"/"FIXME" as part of describing the feature — a clean self-referential edge case, resolved via the contract's `allow` escape hatch rather than by weakening the check.

4 Experimental Setup

4.1 Substrate

We use SWE-bench Lite [Jimenez et al., 2024], a 300-instance subset of SWE-bench covering real GitHub issues across 11 popular Python repositories. Rather than producing new agent rollouts ourselves, we use a real, publicly published, externally produced submission from the official SWE-bench `experiments` repository (`https://github.com/swe-bench/experiments`): run `20240620_sweagent_claude3.5sonnet`, SWE-agent [Yang et al., 2024] driving Claude 3.5 Sonnet. For every instance this run publishes (in a public, unauthenticated Amazon S3 bucket) the agent's full step-by-step trajectory, its final submitted unified diff (`patch.diff`), and a ground-truth evaluation report (`report.json`) produced by princeton-nlp's own SWE-bench evaluation harness, independent of this work. We fetched all three artifacts for all 300 instances; 287 have a usable ground-truth report (the remaining 13 either never produced a generation at all, 11 instances, or are missing a report for unrelated reasons, 2 instances — see `benchmark/data/fetch_summary.json`).

Table 1: Arm A (status-quo self-report governance) vs. Arm B (GATEKEEP) on 287 real SWE-bench Lite instances, scored against official SWE-bench ground truth. TP/FP/FN/TN are with respect to predicting `resolved=True`.

Arm	TP	FP	FN	TN	Precision	Recall	F1	Accuracy
A: status-quo baseline	69	218	0	0	0.2404	1.0000	0.3876	0.2404
B: gatekeep	69	214	0	4	0.2438	1.0000	0.3920	0.2544

4.2 Arms

Arm A (status-quo baseline). An instance is "shippable" iff the agent's trajectory contains a non-empty `submission` string and the trajectory's `exit_status` begins with `"submitted"`. This mirrors how most agent harnesses gate today: trust the agent's own completion signal, with no inspection of the artifact's content.

Arm B (gatekeep). An instance is "shippable" iff GATEKEEP's deterministic contract (`benchmark/contracts/swebench_patch.yml`) passes against that instance's `patch.diff`: the patch must exist, be at least 30 bytes, be a syntactically valid unified diff touching at least one non-test file, and contain no placeholder/error markers (`NotImplementedError`, `FIXME`, etc.) on any line the patch *adds*. Both arms make their decision before consulting `report.json`.

4.3 Scoring

We treat "shippable" as a binary classifier for the real ground-truth label `resolved` from `report.json`, and report precision, recall, F1, and accuracy for both arms over the 287 instances with ground truth, plus a headline operational number: of the 218 instances that were *not* actually resolved ("doomed" deliverables), how many did each arm let through to the costly test-execution stage versus catch for free beforehand.

5 Results

Table 1 reports the full confusion-matrix metrics for both arms on the 287 SWE-bench Lite instances with ground truth, computed by `benchmark/scripts/run_ab.py` and saved verbatim to `benchmark/results/ab_summary.json` and `ab_raw_rows.json` (one row per instance, with the exact check output that produced each verdict).

Out of 287 instances, 69 were genuinely resolved and 218 were not. **Arm A has zero discriminative power**: TN $= 0$, meaning that among *every* instance the agent's own harness considered cleanly submitted, not a single one was flagged as suspect, regardless of whether the underlying patch was sound. This is the quantitative version of "the agent always says it's done."

Arm B catches 4 of the 218 doomed deliverables for free, before any test suite runs, with **zero additional false negatives** (recall stays at 1.00 – it never incorrectly flags a deliverable that was actually resolved). The four catches break down as: three instances where the agent's submitted patch touched *only* test files (e.g. `django__django-16229`, whose patch modifies `tests/custom_test_runner.py`, `tests/test_arrayfield_admin.py`, and two other test files but no source file at all — a patch that, by construction, cannot fix the underlying issue), and one instance (`sympy__sympy-22005`) where the submitted diff adds a `raise NotImplementedError(...)` for an unsupported code path.

The deterministic check itself is fast: scoring all 300 local instances (running the full contract against each instance's artifact directory) takes 0.157 s wall-clock total, or 0.52 ms per instance, on a single consumer laptop core, with no network call.

5.1 Where the four catches come from, qualitatively

We manually inspected the four instances Arm B flags and Arm A does not (Table 2). All four are genuine: none is an artifact of an overly aggressive check rule (we verified this by also inspecting, and ruling out as false positives, two close calls discussed in Section 6).

Table 2: The four doomed deliverables GATEKEEP flags that the status-quo baseline does not.

Instance	Reason flagged
`django__django-16229`	Patch touches only test files (4 files, 0 source files).
`django__django-16408`	Patch touches only `tests/known_related_objects/tests.py`.
`pytest-dev__pytest-7168`	Patch touches only `test_repr_error.py`.
`sympy__sympy-22005`	Patch adds `raise NotImplementedError(...)` for an unhandled case.

6 Limitations and Honest Failure Modes

We report two real bugs found and fixed during this experiment, because a governance tool's credibility depends on being honest about its own false positives.

Bug 1: substring test-path matching. An early version of the "forbid test-only patches" rule classified a path as a test file if the substring `"test"` appeared anywhere in the path. This misclassified `src/_pytest/assertion/rewrite.py` (a real source file in the `pytest` project) as a test file purely because `"pytest"` contains the substring `"test"`, producing a false negative on instance `pytest-dev__pytest-11143` (a resolved instance that Arm B would have incorrectly flagged as unsound). We fixed this with a path-component-aware predicate (a component must equal `test`/`tests`, or the filename stem must start with `test_` or end with `_test`) and added a regression test.

Bug 2: removed-line placeholder false positives. The placeholder-text scanner initially scanned the entire diff text, including context and *removed* lines. This flagged `django__django-12286` (a resolved instance) because its patch *deletes* two pre-existing `# FIXME` comments as part of removing dead code — the agent did not introduce a placeholder, it removed two. We added a `diff_added_lines_only` mode that scans only lines the patch adds, and a regression test asserting a removed `FIXME` does not trigger a failure while an added one still does.

Both bugs were caught by running the real experiment end-to-end against real data and inspecting disagreements between gatekeep and ground truth — not by code review alone. We consider this evidence in favor of the project's own thesis: deterministic checks are only as trustworthy as the checks you write, and validating them against real outcomes (not just unit tests of the checker's own logic) is necessary.

What deterministic checks cannot do. Even after both fixes, Arm B's precision (0.244) is far below 1.0: 214 of its 283 "shippable" calls are deliverables that are not actually resolved. This is expected and by design — a syntactically valid, non-placeholder, source-touching patch can still be semantically wrong. GATEKEEP verifies a necessary, not sufficient, condition for deliverable soundness. It is not a replacement for running the actual test suite or for human review; it is a cheap, instant pre-filter that catches the cases where the deliverable does not even have the right *shape* to be correct, before paying the cost of deeper verification.

One more honest caveat. Our `NotImplementedError` pattern flags any added line containing that token, including legitimate exception-handling code (e.g. `except NotImplementedError:`, `assertRaises(NotImplementedError)`) alongside genuine unfinished stubs. In our data this did not produce a false positive on a resolved instance (the one match, `sympy__sympy-22005`, is a true positive for "doomed" regardless of the specific line that triggered it), but a contract author relying on this pattern in a codebase with heavy exception-handling idioms should expect lower precision on that specific check and may want to narrow the pattern (e.g. to `raise NotImplementedError` only, excluding `except`/`assertRaises` contexts).

7 Discussion

The headline finding of this paper is not that GATEKEEP is a large improvement over the baseline in aggregate F1 (0.392 vs. 0.388 — a modest difference, because most of the discriminative work on this particular dataset would require semantic, not structural, verification). The headline finding is that **the status-quo baseline has *zero* true negatives on real data**: it provides no information beyond "did the agent's own harness claim success," which is exactly the self-report problem this

paper set out to quantify. Any non-zero, zero-false-negative improvement over a classifier with literally no discriminative power is operationally meaningful, because it is pure signal added at effectively zero cost (0.52 ms/instance, no model call, no network, fully reproducible).

We see GATEKEEP as one layer in a larger governance stack, not a complete solution: deterministic structural checks for the necessary conditions (this work), optionally composed with semantic checks (test execution, or an LLM-judge plugin for genuinely soft rubric criteria) for the sufficient conditions. The contribution of this paper is showing that the cheap layer is not vacuous — it catches real, qualitatively inspectable failures on real public data — and that it can be built, tested, and shipped as a small, auditable, dependency-light artifact that a long-horizon agent task owner can adopt in minutes.

8 Reproducibility

All numbers in this paper are produced by code in the public repository accompanying this submission. `benchmark/scripts/fetch_swebench_run.py` re-downloads the 300 instances' raw artifacts from the public, unauthenticated S3 bucket; `benchmark/scripts/distill_trajs.py` extracts the trajectory summary used for Arm A; `benchmark/scripts/run_ab.py` reproduces Table 1 and both raw-row and summary JSON files exactly. The GATEKEEP engine itself has 20 unit/regression/parity tests (`gatekeep/tests/`), all passing at submission time, including a dedicated regression test for each bug in Section 6. No number in this paper was generated by, or required, an LLM API call.

References

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* International Conference on Learning Representations (ICLR), 2024.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering.* arXiv:2405.15793, 2024.